

Sesión 10. Redes neuronales convolucionales

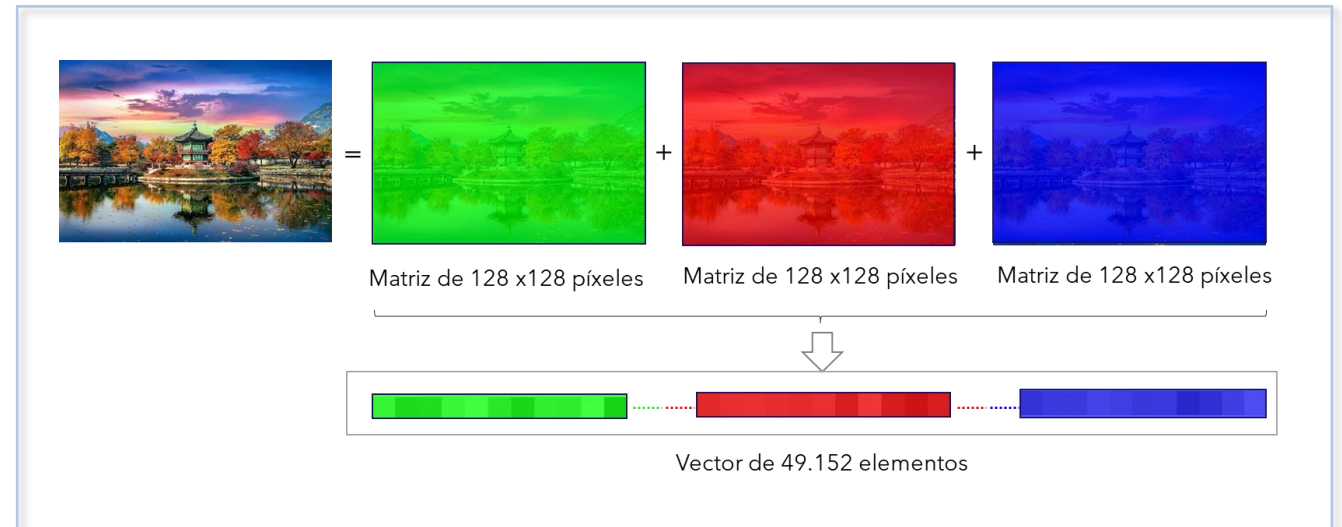
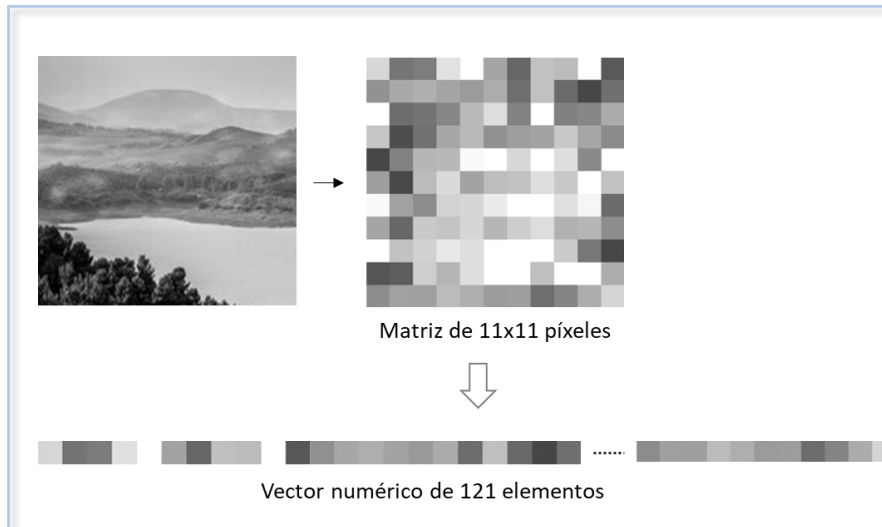
- ¿Cómo procesar las imágenes?
- Redes convolucionales. Conceptos.
- Algunas tareas de procesamiento de imágenes y arquitecturas de CNN.
- Aspectos de entrenamiento.

¿Cómo procesar las imágenes?

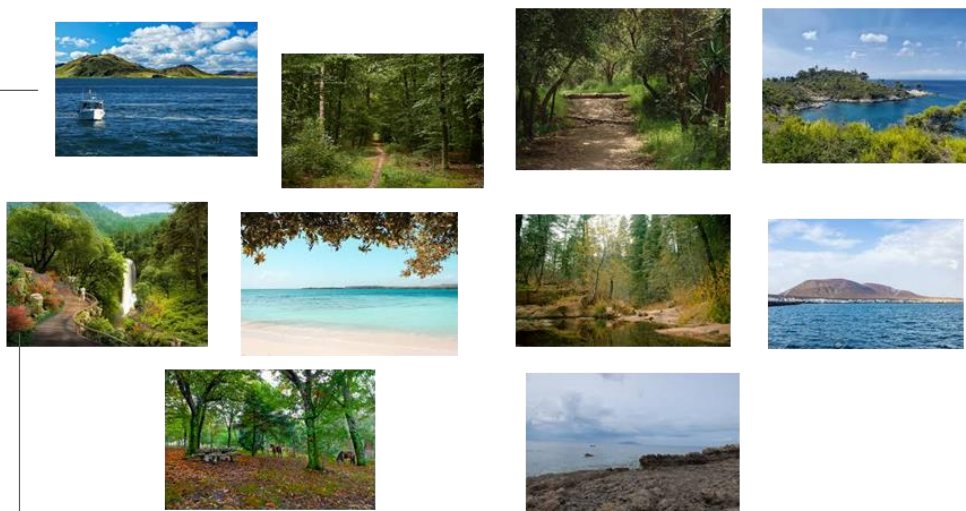
Una imagen se representa como una matriz en la que cada elemento se conoce como píxel. Cada píxel contiene un valor numérico en un rango que dependerá del tipo de imagen. Por ejemplo:

Imagen	Valores de píxel
Blanco y negro (Bitmap)	0 (negro), 1 (blanco)
Escala de grises	[0, 255] (en caso de 8 bits por píxel)
A color (por ejemplo, RGB)	[0,255] por cada canal (8 bits por canal)

- Representar cada imagen como un vector d-dimensional, donde d = número de píxeles de cada canal.



Ejemplo: clasificación de imágenes.



Conjunto de datos:
N imágenes de
resolución 128x128

Preparación

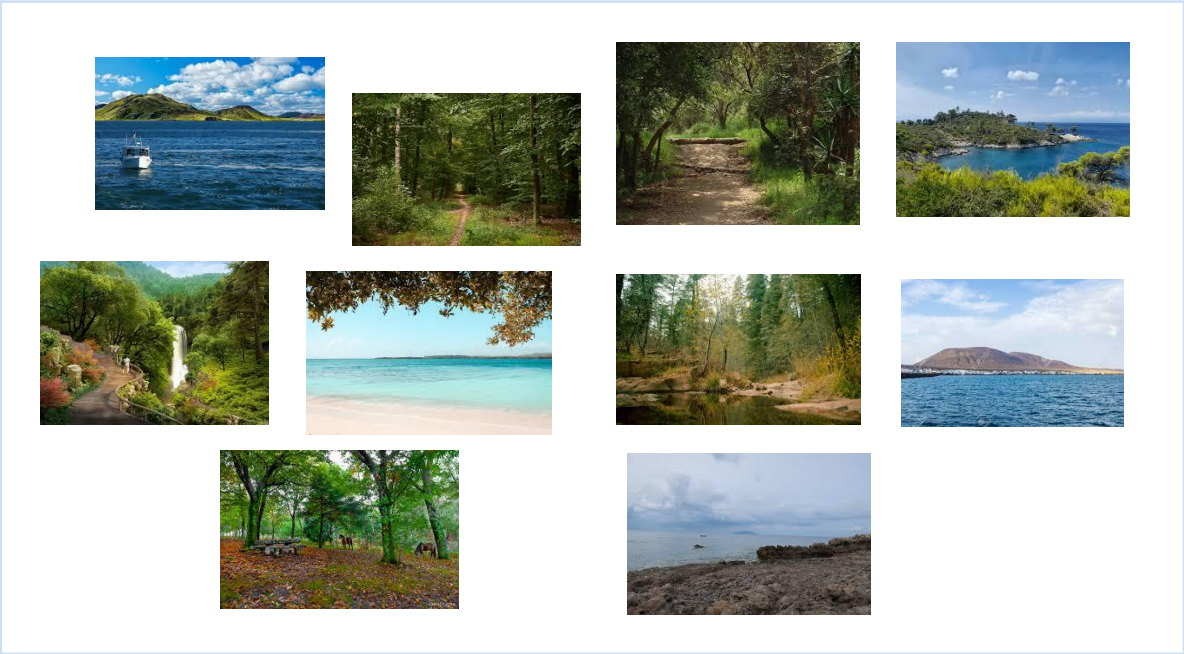
[illegible]

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense

# Definición del modelo
model = Sequential([
    Flatten(input_shape=(128, 128, 3)), # Aplana la imagen
    Dense(512, activation='relu'),        # 1ª capa oculta
    Dense(256, activation='relu'),        # 2ª capa oculta
    Dense(128, activation='relu'),        # 3ª capa oculta
    Dense(1, activation='sigmoid')        # Capa de salida (binaria)
])

# Compilación del modelo
model.compile(
    optimizer='adam',
    loss='binary_crossentropy', # para clasificación binaria
    metrics=['accuracy']
)
```

- Construir o derivar atributos para caracterizar las imágenes.

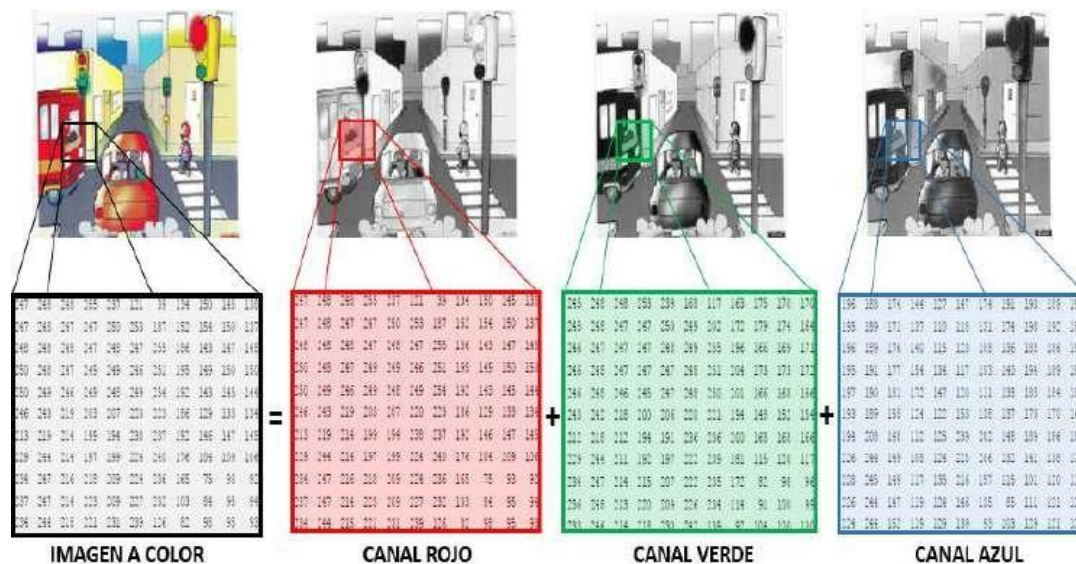


Texture	Mean	Standard deviation	R (normalized)	3rd moment	Uniformity	Entropy
Smooth	82.64	11.79	0.002	-0.105	0.026	5.434
Coarse	143.56	74.63	0.079	-0.151	0.005	7.783
Regular	99.72	33.73	0.017	0.750	0.013	6.674

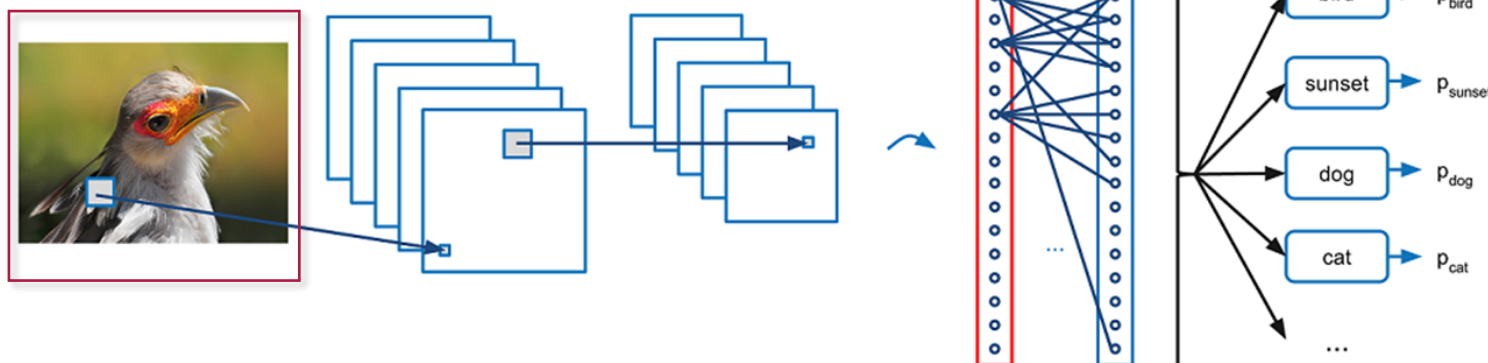


MediaR	MediaG	MediaB	VarianzaR	VarianzaG	VarianzaB		Tipo
							Marino
							Bosque
							Bosque
							Marino

- Representar cada imagen como una matriz de píxeles por cada canal.



En redes neuronales convolucionales:



- Diseñadas para trabajar con entradas estructuradas en forma de grid (cuadrícula), que tienen dependencias espaciales (por ejemplo, imágenes).
- La mayoría de las aplicaciones de redes neuronales convolucionales se centran en imágenes, pero también se pueden usar para datos temporales, espaciales y espacio-temporales.
- Una característica importante de las redes neuronales convolucionales es la operación de **convolución**.
- Las redes neuronales convolucionales se definen entonces como redes que usan la operación convolucional en al menos una capa.

Redes convolucionales. Conceptos.

¿Qué es una convolución?

- El proceso de convolución mide el impacto de una señal cuando se integra con otra señal.

$$s(t) = (x \circledast w)(t) = \int_{-\infty}^{\infty} x(k)w(t-k)\partial k$$

- En procesamiento de imágenes, la convolución se define como el tratamiento de una matriz por otra, conocida como kernel o filtro.
- Se utilizan diferentes filtros en forma de matrices de convolución, para lograr diferentes resultados y efectos.
- Los tamaños de los kernels pueden variar, por ejemplo, de 5x5 o 3x3.
- El filtro examina, sucesivamente, cada píxel de la imagen. Para cada uno de ellos, se multiplica el valor de este píxel y el valor de los circundantes por el valor correspondiente del kernel (element-wise producto o Hadamard producto)
- Esto produce otra matriz (imagen) con las características deseadas.

Matriz de entrada

35	40	41	45	50
40	40	42	46	52
42	46	50	55	55
48	52	56	58	60
56	60	65	70	75

Kernel

-2	-1	0
-1	1	1
0	1	2

*

=

Matriz de salida

78		

$$(-2 \times 35) + (-1 \times 40) + (0 \times 41) + (-1 \times 40) + (1 \times 40) + (1 \times 42) + (0 \times 42) + (1 \times 46) + (2 \times 50) = 78$$

35	40	41	45	50
40	40	42	46	52
42	46	50	55	55
48	52	56	58	60
56	60	65	70	75

-2	-1	0
-1	1	1
0	1	2

*

=

78	87	

$$(-2 \times 40) + (-1 \times 41) + (0 \times 45) + (-1 \times 40) + (1 \times 42) + (1 \times 46) + (0 \times 62) + (1 \times 50) + (2 \times 55) = 87$$

35	40	41	45	50
40	40	42	46	52
42	46	50	55	55
48	52	56	58	60
56	60	65	70	75

-2	-1	0
-1	1	1
0	1	2

*

=

Al recorrer toda la imagen:

78	87	94
98	283	108
120	125	127

Ejemplos:

- Operador de Sobel para detección de bordes.

Imagen de entrada



*

-1	0	1
-2	0	2
-1	0	1

=

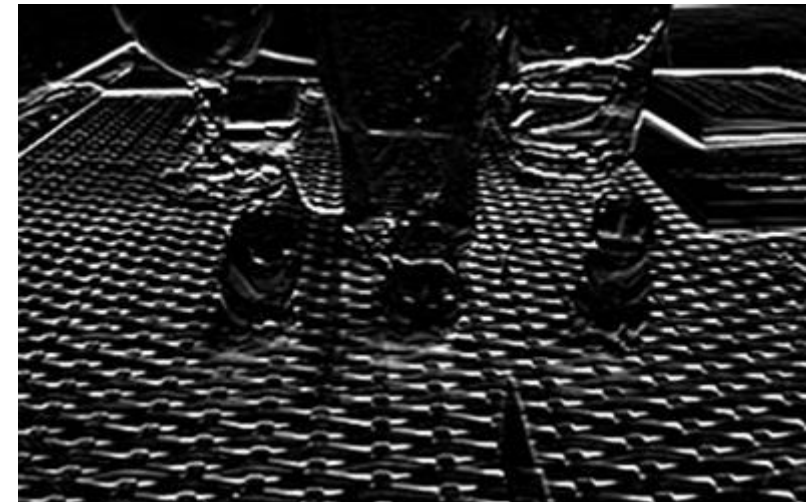
Imagen de salida



*

1	2	1
0	0	0
-1	-2	-1

=



- Operador para enfocar.

Imagen de entrada



*

0	-1	0
-1	5	-1
0	-1	1

=



Imagen de salida

- Operador para repujar.

Imagen de entrada



*

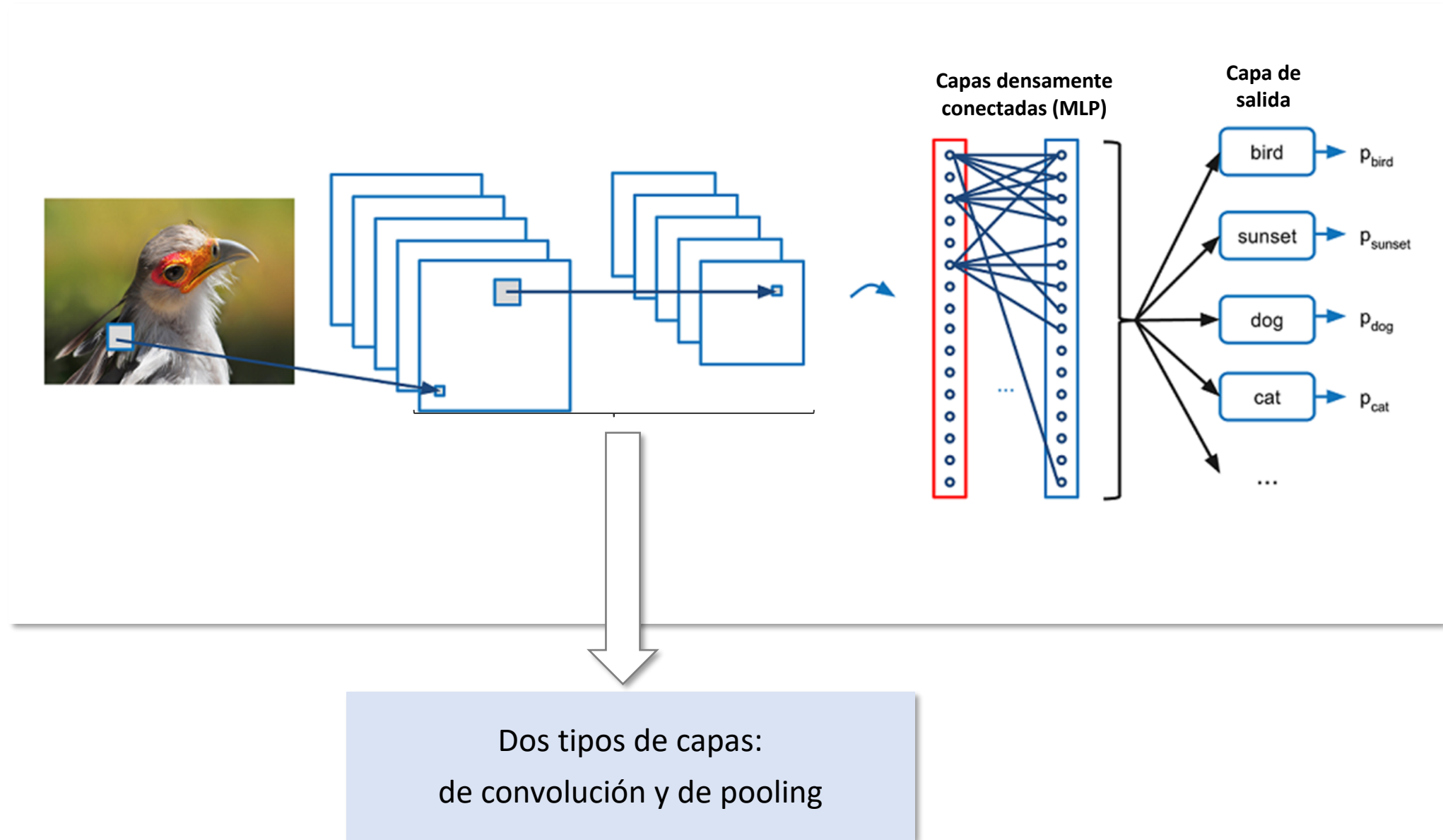
-2	-1	0
-1	1	1
0	1	2

=

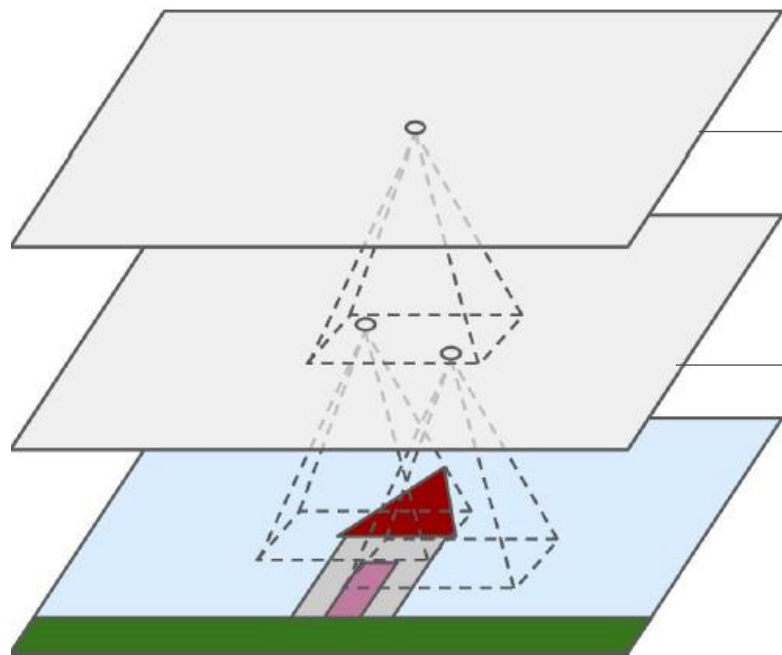


Imagen de salida

Estructura general de una red convolucional



1. Capa convolucional



A su vez, cada neurona en la segunda capa está conectada solo a las neuronas ubicadas dentro de un pequeño rectángulo en la primera capa.

Neuronas en la primera capa no están conectadas a cada píxel en la imagen de entrada, sino solo a los píxeles en sus campos receptivos.

Esta arquitectura permite que la red se concentre en las operaciones de bajo nivel en la primera capa oculta. Luego, en las operaciones de nivel superior en la siguiente capa oculta, y así sucesivamente.

Geron, A. (2022). Hands-On Machine Learning with Scikit-Learn and TensorFlow. Concepts, Tools, and Techniques to Build Intelligent Systems. O'Reilly Media, Inc

Bengio, Y. , Courville, A., Goodfellow, I. (2016). Deep learning. Adaptive computation and machine learning. The MIT Press.

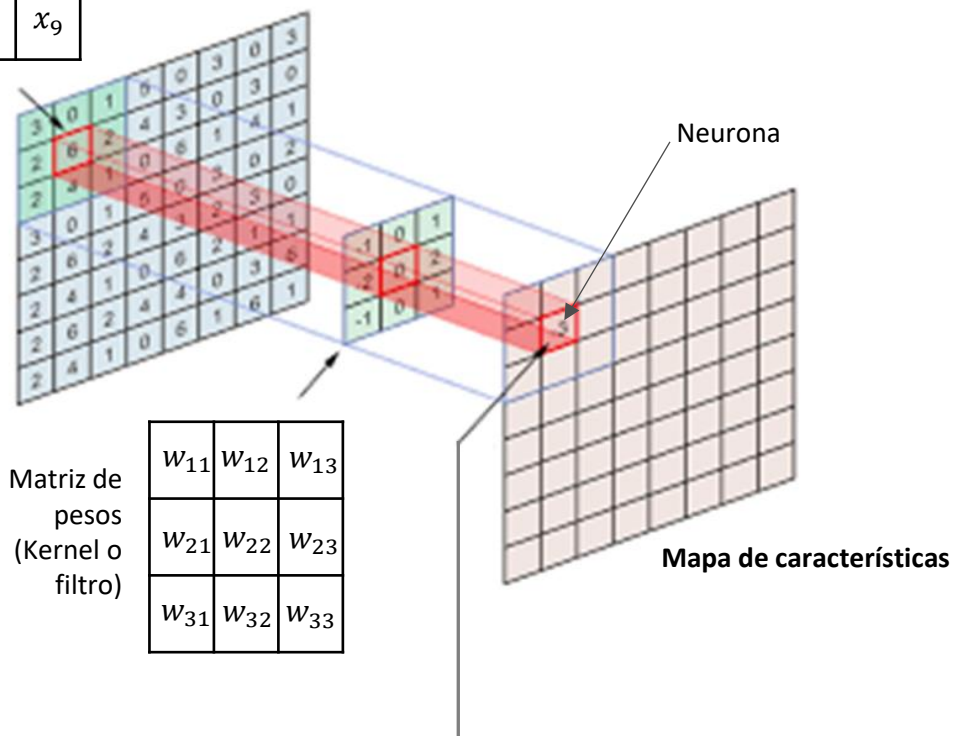
Ejemplo:



¿Cómo funciona?

Entrada

x_1	x_2	x_3
x_3	x_5	x_6
x_7	x_8	x_9



$$f_{\text{activación}}(w_{11} \times x_1 + w_{12} \times x_2 + \dots + w_{22} \times x_5 + \dots + w_{33} \times x_9)$$

Todas las neuronas de una capa comparten el mismo conjunto de pesos (implementan el mismo filtro)

Imagen de entrada

x_1	x_2	x_3	x_4
x_5	x_6	x_7	x_8
x_9	x_{10}	x_{11}	x_{12}
x_{13}	x_{14}	x_{15}	x_{16}

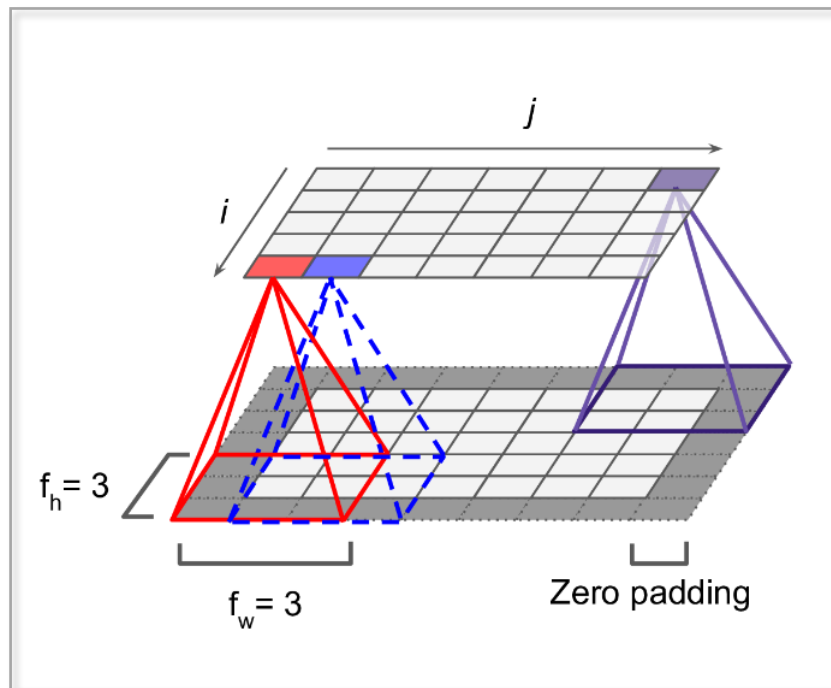
Filtro

w_{11}	w_{12}
w_{21}	w_{22}

$w_{11}x_1 + w_{12}x_2 + w_{21}x_5 + w_{22}x_6$	$w_{11}x_2 + w_{12}x_3 + w_{21}x_6 + w_{22}x_7$	$w_{11}x_3 + w_{12}x_4 + w_{21}x_7 + w_{22}x_8$
$w_{11}x_5 + w_{12}x_6 + w_{21}x_9 + w_{22}x_{10}$	$w_{11}x_6 + w_{12}x_7 + w_{21}x_{10} + w_{22}x_{11}$	$w_{11}x_7 + w_{12}x_8 + w_{21}x_{11} + w_{22}x_{12}$
$w_{11}x_9 + w_{12}x_{10} + w_{21}x_{13} + w_{22}x_{14}$	$w_{11}x_{10} + w_{12}x_{11} + w_{21}x_{14} + w_{22}x_{15}$	$w_{11}x_{11} + w_{12}x_{12} + w_{21}x_{15} + w_{22}x_{16}$

Mapa de características

a) **Padding**: cantidad de píxeles añadidos a la imagen para su procesamiento por un filtro convolucional.



Matriz de entrada

35	40	41	45	50
40	40	42	46	52
42	46	50	55	55
48	52	56	58	60
56	60	65	70	75

Zero padding

0	0	0	0	0	0	0
0	35	40	41	45	50	0
0	40	40	42	46	52	0
0	42	46	50	55	55	0
0	48	52	56	58	60	0
0	56	60	65	70	75	0
0	0	0	0	0	0	0

Kernel

-2	-1	0
-1	1	1
0	1	2

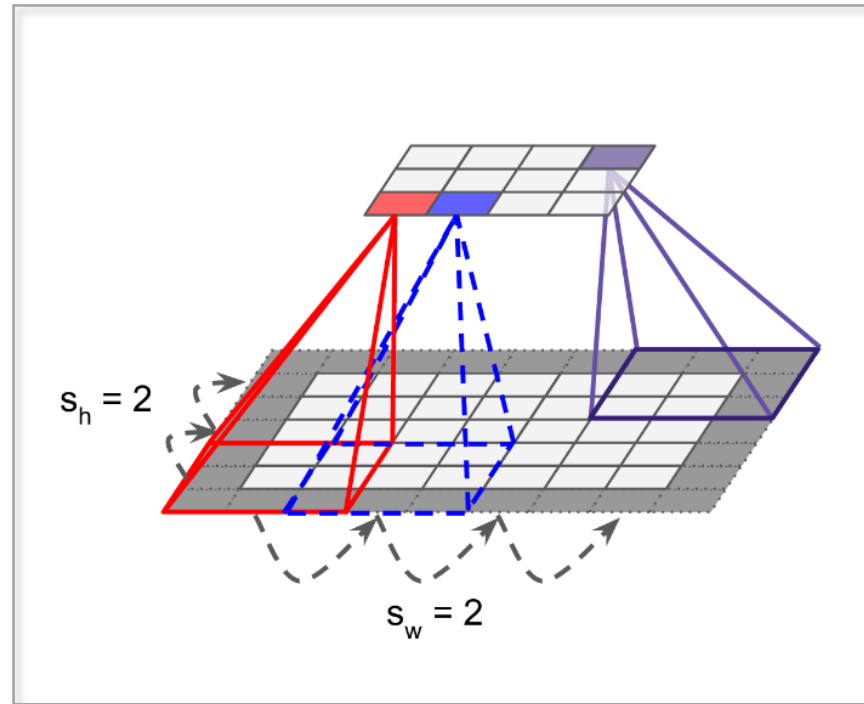
*

=

Matriz de salida

195				
	78	87	94	
	98	283	108	
	120	125	127	

b) **Stride**: tamaño del paso al deslizar el filtro convolucional sobre la imagen de entrada.



Matriz de entrada

35	40	41	45	50
40	40	42	46	52
42	46	50	55	55
48	52	56	58	60
56	60	65	70	75

Kernel

-2	-1	0
-1	1	1
0	1	2

*

Matriz de entrada

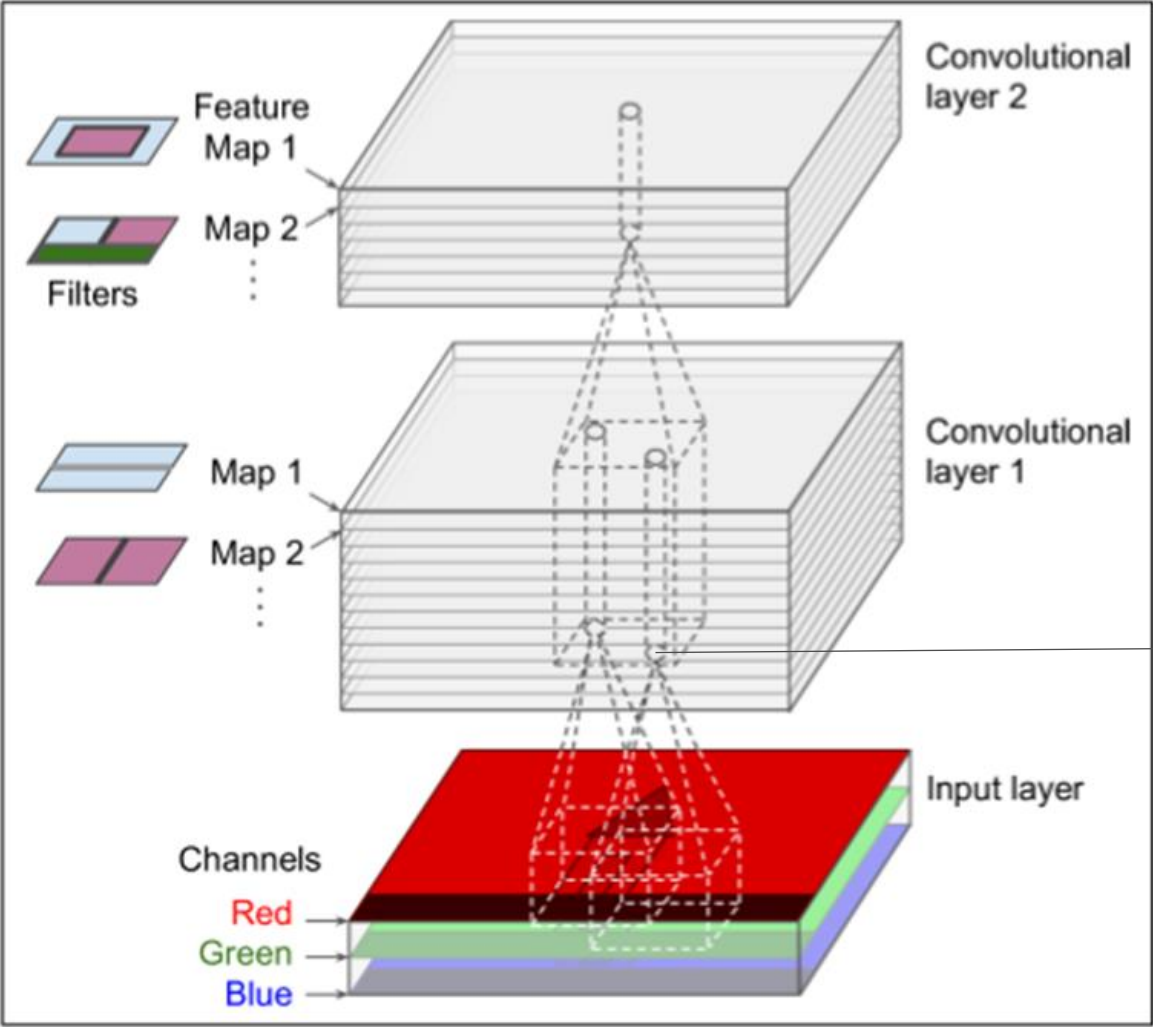
35	40	41	45	50
40	40	42	46	52
42	46	50	55	55
48	52	56	58	60
56	60	65	70	75

Kernel

-2	-1	0
-1	1	1
0	1	2

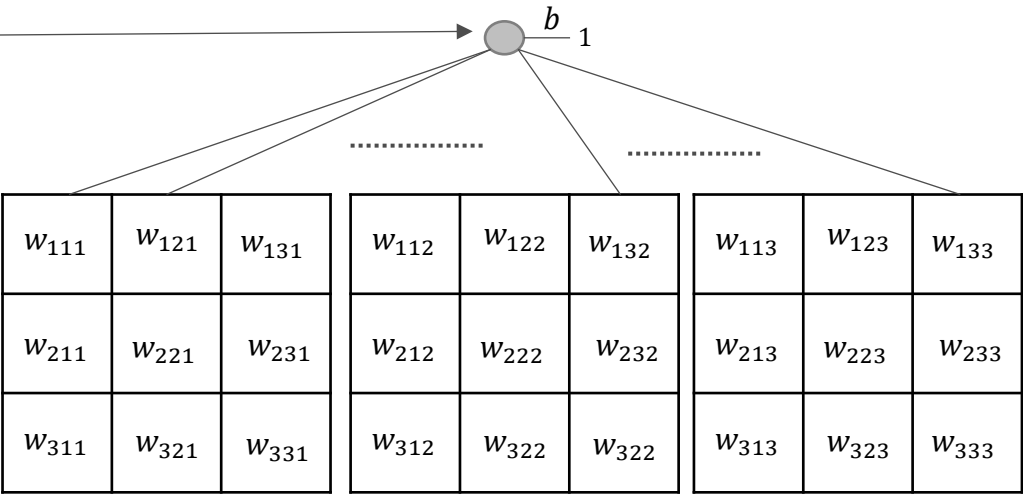
*

Pero,



Cada capa convolucional= volumen

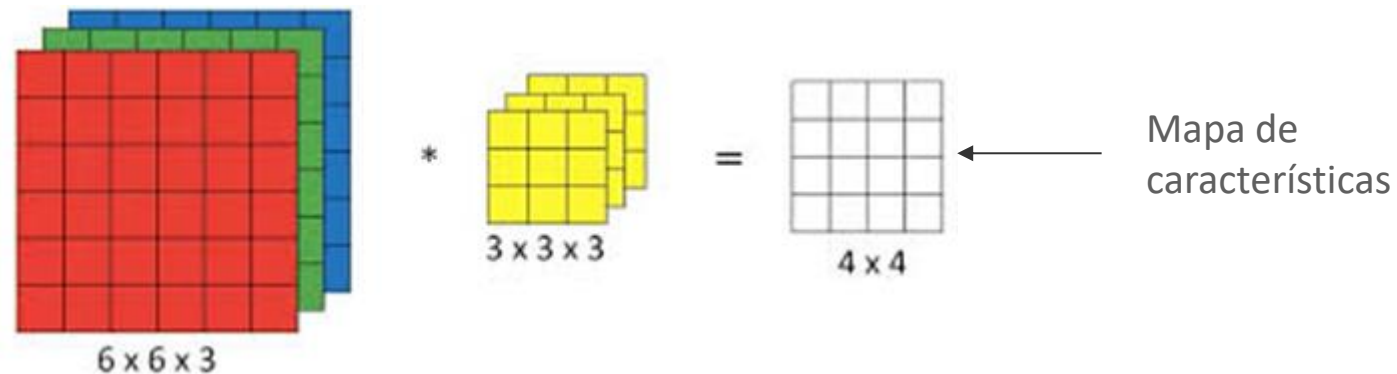
$$Capa = \textit{Altura} \times \textit{Ancho} \times \textit{Profundidad} = L_q \times B_q \times d_q$$



- Cada mapa de características se obtienen al aplicar un filtro o kernel a la capa anterior.
- La profundidad del filtro es la misma de la capa a la cual se aplica.
- El tamaño del filtro será:

$$\text{Filtro} = \text{Altura} \times \text{Ancho} \times \text{Profundidad} = F_q \times F_q \times d_q$$

- La operación de convolución coloca el filtro en cada posible posición de la imagen (o capa oculta) y realiza un producto interno entre los parámetros $F_q \times F_q \times d_q$ y el volumen de entrada (también de tamaño $F_q \times F_q \times d_q$):



Ejemplo:

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Filtro

-1	-1	1
0	1	-1
0	1	1

$w \cdot x = 308$

1	0	0
1	-1	-1
1	0	-1

$w \cdot x = -498$

0	1	1
0	1	0
1	-1	1

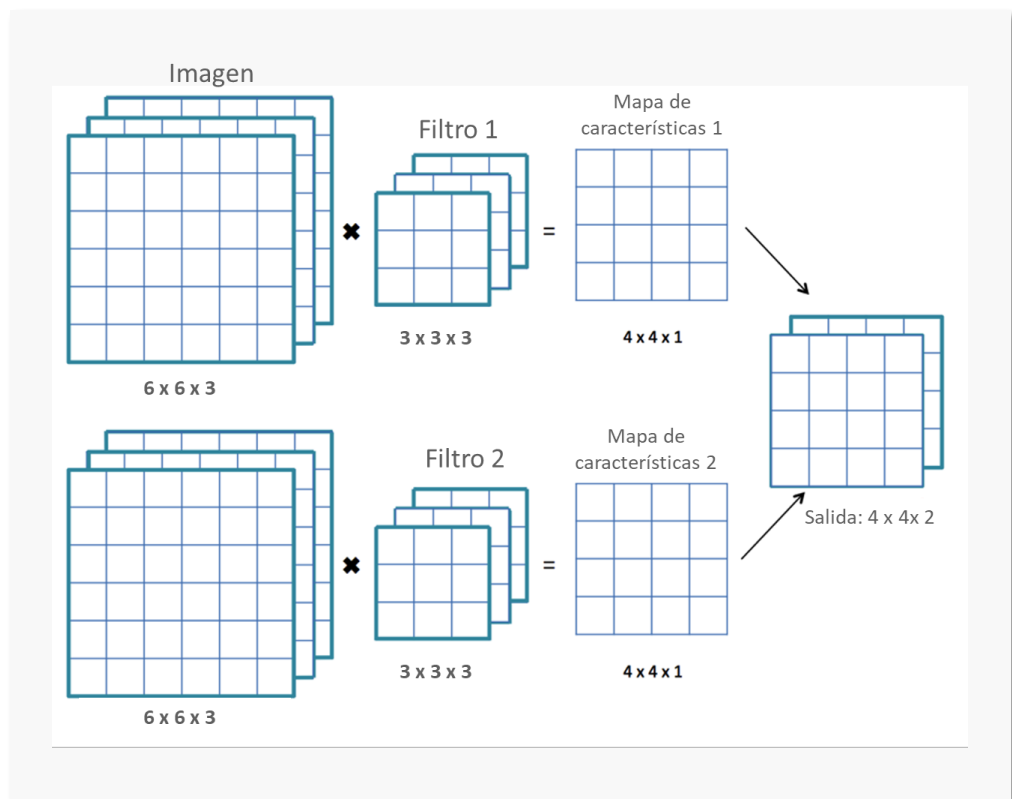
$w \cdot x = 164$

$Bias = 1$

$f(308 - 498 + 164 + 1)$

Mapa de características

- Una capa convolucional puede tener más de un mapa de características; cada uno asociado a un filtro diferente con su conjunto de parámetros.
- Entonces, el número de características de una capa depende del número de filtros utilizados en la operación de convolución.
- El número de filtros utilizados en cada capa controla la capacidad del modelo, es decir, el número de parámetros.



Capa = Altura × Ancho × Profundidad = $L_q \times B_q \times d_q$

Filtro = Altura × Ancho × Profundidad = $F_q \times F_q \times d_q$

- Cada filtro tratará de capturar un tipo particular de patrón espacial en una pequeña región de la imagen (campo receptivo).
- Un filtro p en la capa q tiene parámetros que se pueden representar por un tensor de tres dimensiones:

$$W^{(p,q)} = [w_{ijk}^{(p,q)}] \quad \text{donde} \quad \begin{cases} i = \text{altura} \\ j = \text{ancho} \\ k = \text{profundidad} \end{cases}$$

- El mapa de características está representado por un tensor de tres dimensiones:

$$H^q = [h_{ijk}^{(q)}] \quad \text{si } q = 1 \Rightarrow \text{capa de entrada}$$

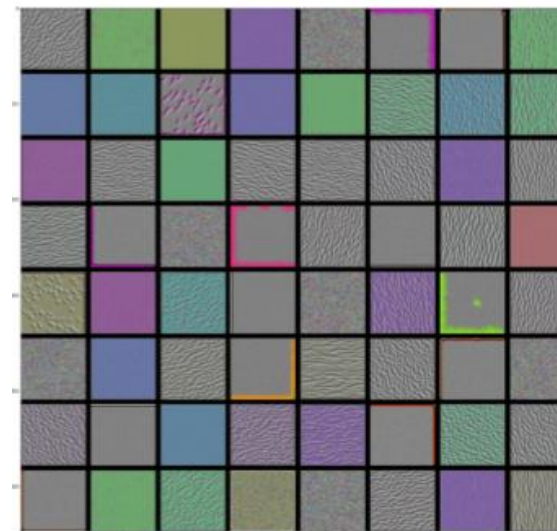
- Las operaciones convolucionales de la capa q a la capa $(q + 1)$ se define como:

$$h_{ijp}^{(q+1)} = \sum_{r=1}^{F_q} \sum_{s=1}^{F_q} \sum_{k=1}^{d_q} w_{rst}^{(q)} \times h_{i+r-1,j+s-1,k}^{(q)}$$

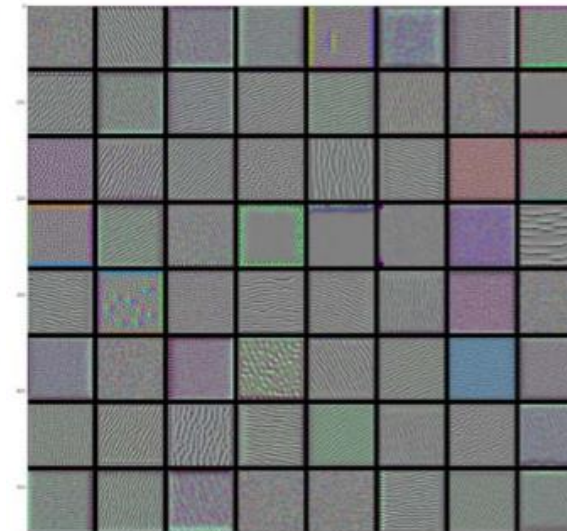
De esta forma....

- Se aprenden patrones locales, los cuales son invariantes a la traslación y están organizados en una jerarquía.
- La profundidad de cada capa convolucional depende del número de filtros de convolución que se aplican sobre la capa anterior.
- Parámetros de una capa de convolución: tamaño del filtro y número de filtros (profundidad).
- Los filtros codifican aspectos específicos de los datos, y cada uno produce como salida un mapa de características.

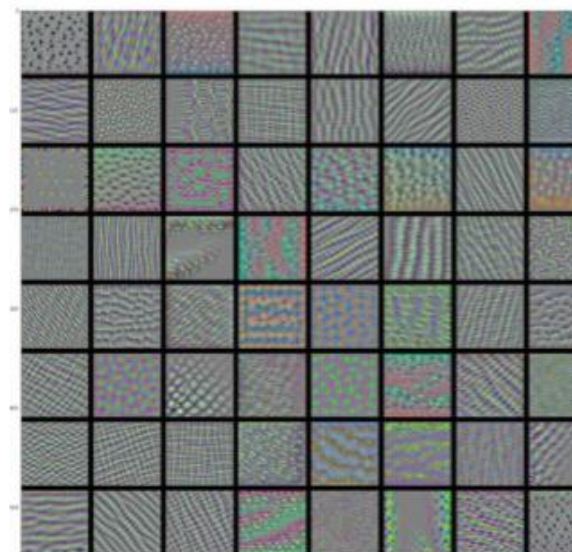
Capa convolucional 1



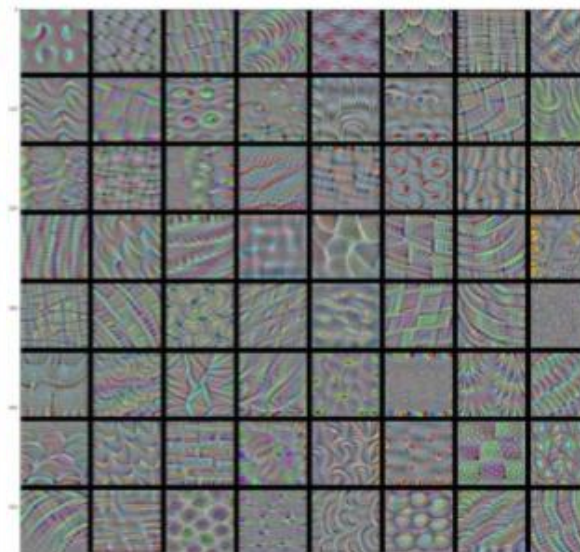
Capa convolucional 2



Capa convolucional 3

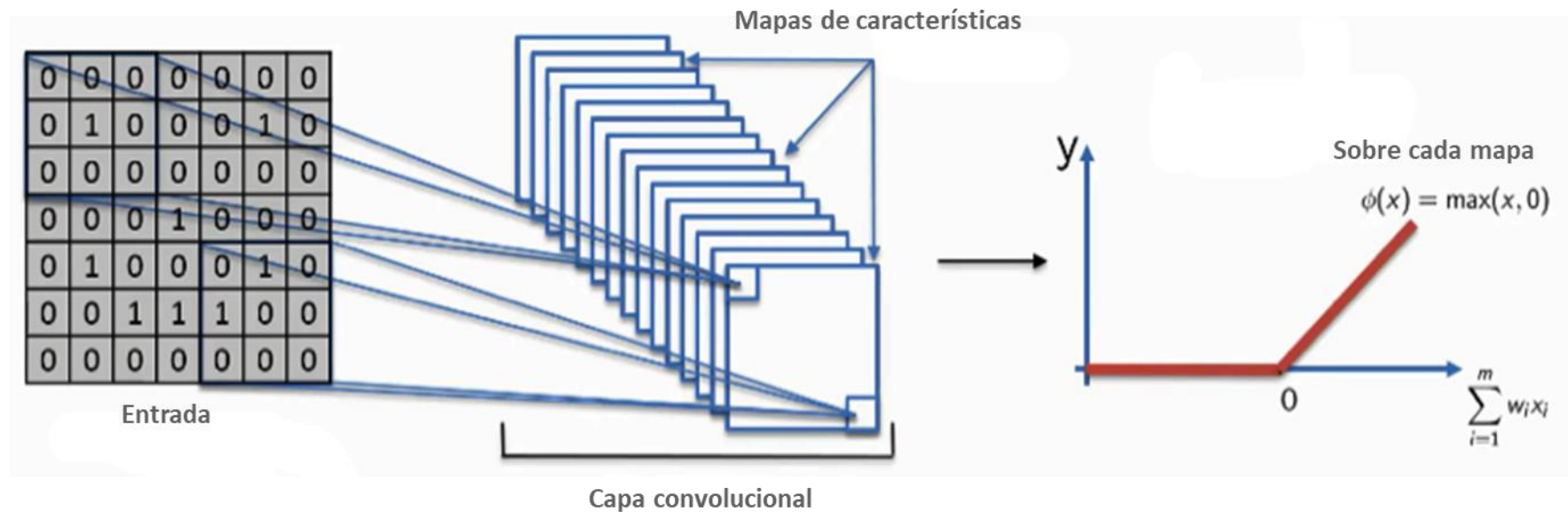


Capa convolucional 4



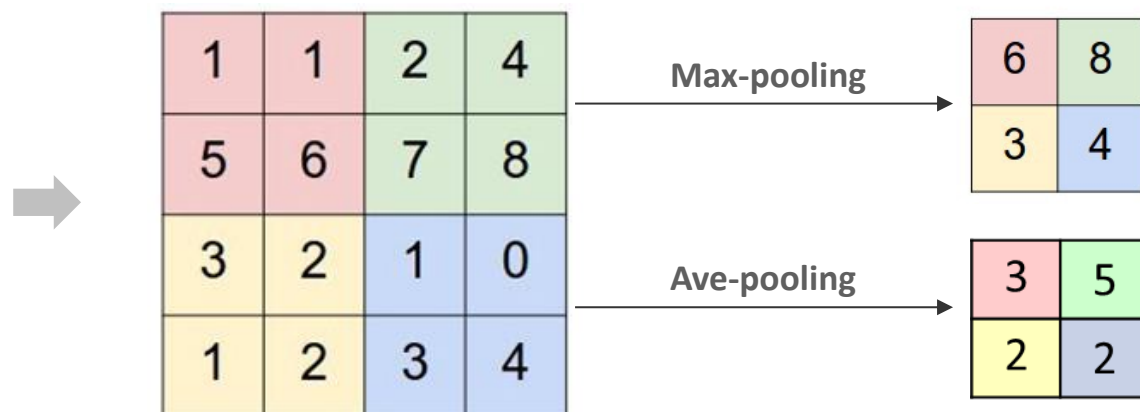
En resumen,

- Todas las neuronas en un mapa comparten el mismo conjunto de pesos, lo cual reduce el número de parámetros del modelo.
- Los filtros no se definen manualmente, se aprenden durante el entrenamiento de la red.
- Para resolver efectos de borde, se utiliza **padding**. Si “same” se realiza el padding de tal manera de producir un mapa del mismo tamaño de la entrada (se agregan ceros a los bordes). Si “valid” solo se utilizan las posiciones válidas.
- Es posible modificar la distancia entre dos sucesivas aplicaciones del filtro, a través del parámetro **stride**.
- Una vez determinada la entrada a cada neurona, se aplica la función de activación (ReLU, Sigmoide, ...)

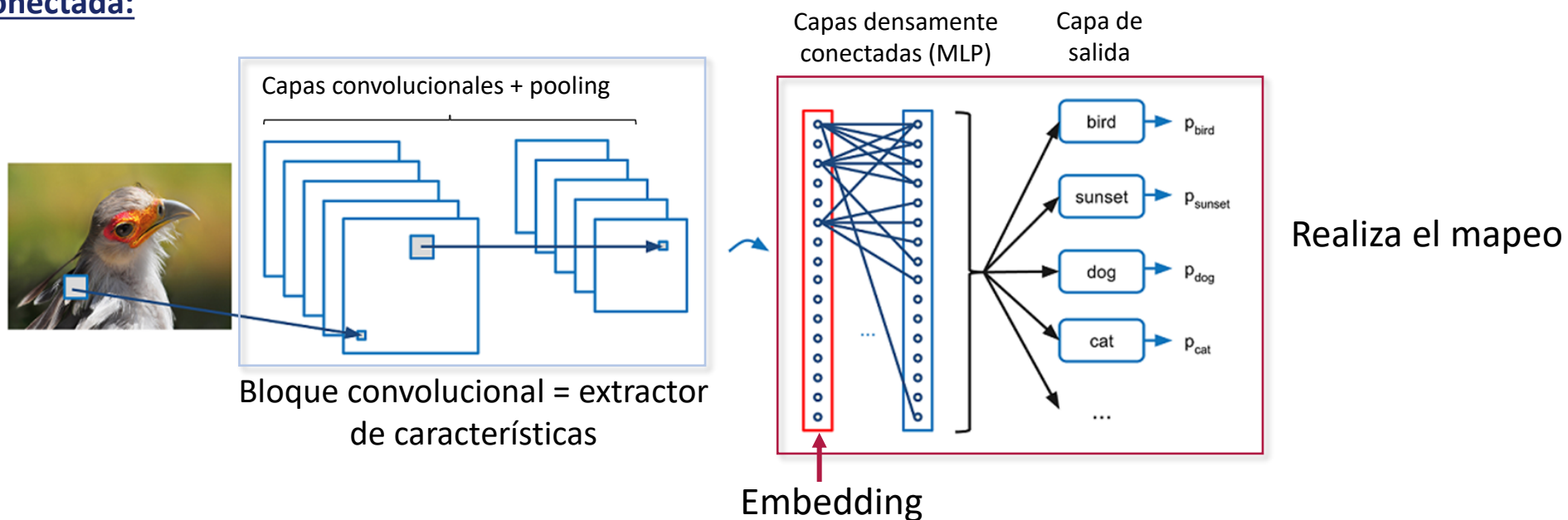


2. Capa de pooling:

- Objetivo: reducción de la dimensionalidad. Realiza un submuestreo.
- Cada neurona en esta capa está conectada a un número limitado de neuronas de la capa anterior (campo receptivo).
- Estas neuronas no tienen pesos asociados; agregan las entradas usando una función, como el máximo o el promedio.



3. Capa totalmente conectada:



Ejemplo. Se quiere resolver un problema relacionado con la identificación temprana de lesiones malignas en la piel a partir de imágenes.

Las imágenes son a color, de 128 x 128 píxeles, y están asociadas a dos clases: maligna y benigna. Se decide utilizar una red convolucional, con dos capas convolucionales. La primera se compone de 64 mapas de características y la segunda de 32.

- ¿Cuántos parámetros tiene cada capa convolucional si se utilizan filtros con una altura y ancho de 5?
- ¿De qué tamaño es el vector de características que sirve de entrada a la capa full conectada, si se coloca una capa de pooling de 2x2 después de la última capa convolucional?

Imagen = $n \times m$

f = altura o ancho del Kernel

s = stride



$$Padding = valid \Rightarrow Imagen_{salida} = \frac{(n - f + s)}{s} \times \frac{(m - f + s)}{s}$$

$$Padding = same \Rightarrow Imagen_{salida} = \frac{(n)}{s} \times \frac{(m)}{s}$$

Si:

Imagen = 128x218

f = altura o ancho del Kernel (5)

s = stride (1 o 2)

Si padding = valid, stride = 1

$$Imagen_{salida} = \frac{(n - f + s)}{s} \times \frac{(m - f + s)}{s}$$

$$Imagen_{salida} = \frac{(128 - 5 + 1)}{1} \times \frac{(128 - 5 + 1)}{1} = 124 \times 124$$

Si padding = same, stride = 1

$$Imagen_{salida} = \frac{(n)}{s} \times \frac{(m)}{s}$$

$$Imagen_{salida} = 128 \times 128$$

Si padding = valid, stride = 2

$$Imagen_{salida} = \frac{(n - f + s)}{s} \times \frac{(m - f + s)}{s}$$

$$Imagen_{salida} = \frac{(128 - 5 + 2)}{2} \times \frac{(128 - 5 + 2)}{2} = 62 \times 62$$

Si padding = same, stride = 2

$$Imagen_{salida} = \frac{(n)}{s} \times \frac{(m)}{s}$$

$$Imagen_{salida} = 64 \times 64$$

Si pooling de 2x2: reduce a la mitad (divide cada dimensión por dos)

Algunas tareas para el procesamiento de imágenes

- **Clasificación.** Asignación de una etiqueta de clase a la imagen.
- **Clasificación y localización.** Se determina la categoría de un objeto en la imagen y su localización dentro de ella.
- **Detección de objetos.** Se determina la categoría de varios objetos en la imagen y su localización dentro de ella.
- **Segmentación semántica.** Se asigna una categoría a cada pixel de la imagen.
- **Segmentación de instancias.** Segmentación y detección.

Clasificación



gato

Clasificación y localización



gato

Detección de objetos



perro, perro, gato

Segmentación semántica



Gato, grama, árbol, cielo

Segmentación de instancias



perro, perro, gato

Otras: segmentación panóptica, estimación de pose, generación de imágenes, ...

❑ Clasificación

Conjunto de datos

Entrada

Salida



Árbol



Flor



Árbol



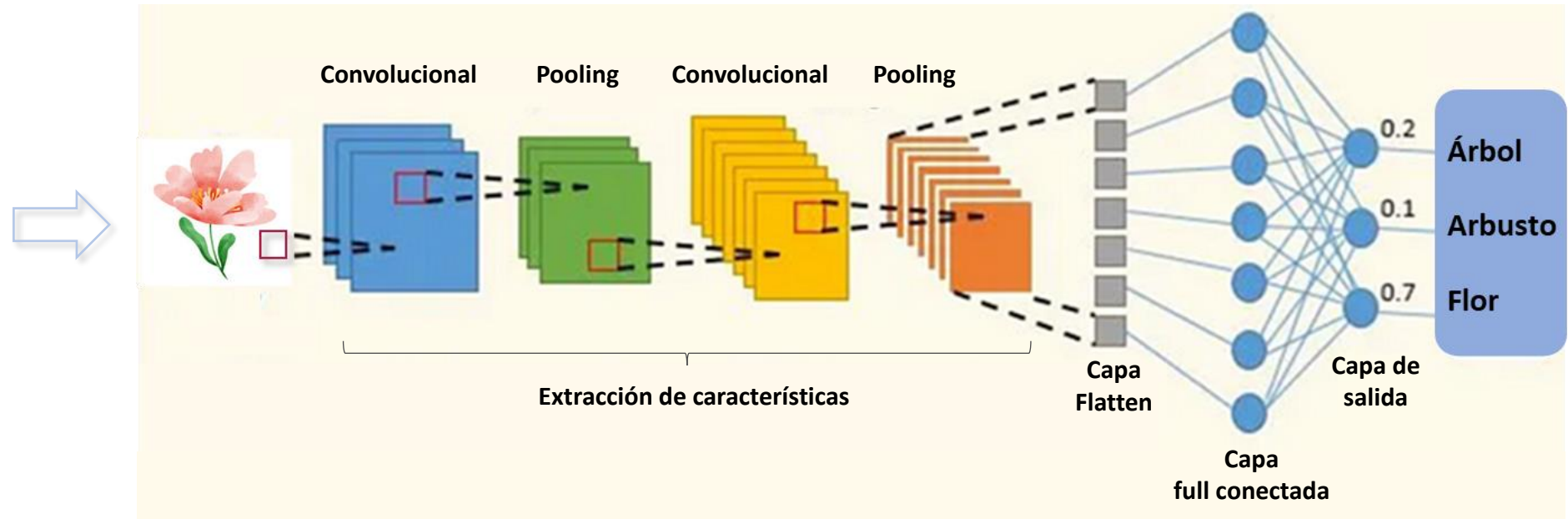
Arbusto

⋮

¿Datos?

- Entrada: Imagen
- Salida: etiqueta de clase

- Capas convolucionales
- Capas de pooling
- Capa de “aplanado” (*flattening*)
- Capas densas

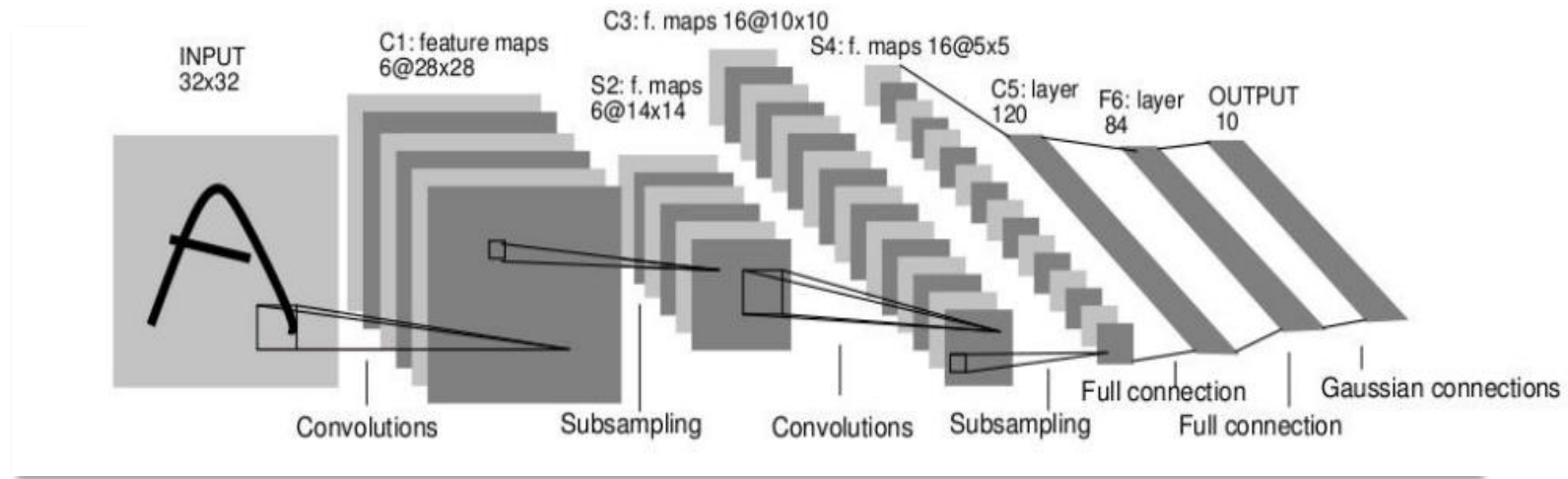


Función de activación de la capa de salida: softmax

Función de costo: entropía cruzada (cross entropy)

□ Arquitecturas de CNN

▪ LeNet-5

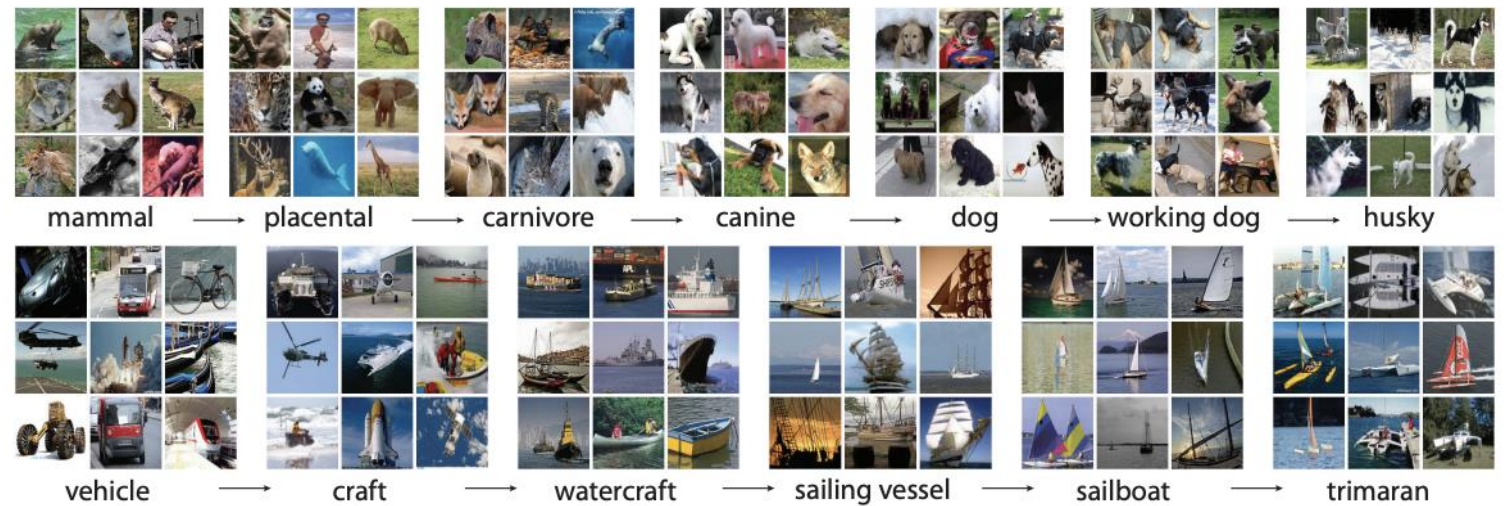


Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully connected	—	10	—	—	RBF
F6	Fully connected	—	84	—	—	tanh
C5	Convolution	120	1×1	5×5	1	tanh
S4	Avg pooling	16	5×5	2×2	2	tanh
C3	Convolution	16	10×10	5×5	1	tanh
S2	Avg pooling	6	14×14	2×2	2	tanh
C1	Convolution	6	28×28	5×5	1	tanh
In	Input	1	32×32	—	—	—

Algunos conjuntos de imágenes:

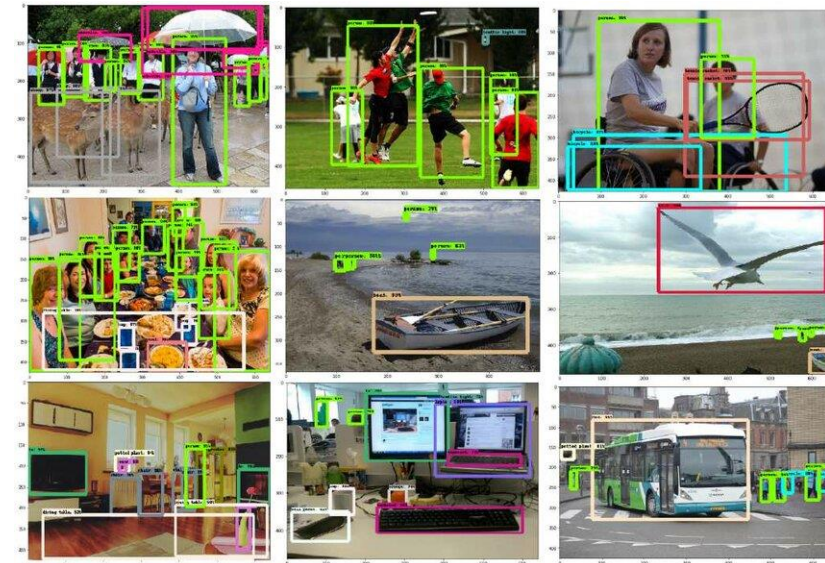
- **ImageNet** (<https://image-net.org/update-mar-11-2021.php>)

- 14 millones de imágenes, 1000 clases
- ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

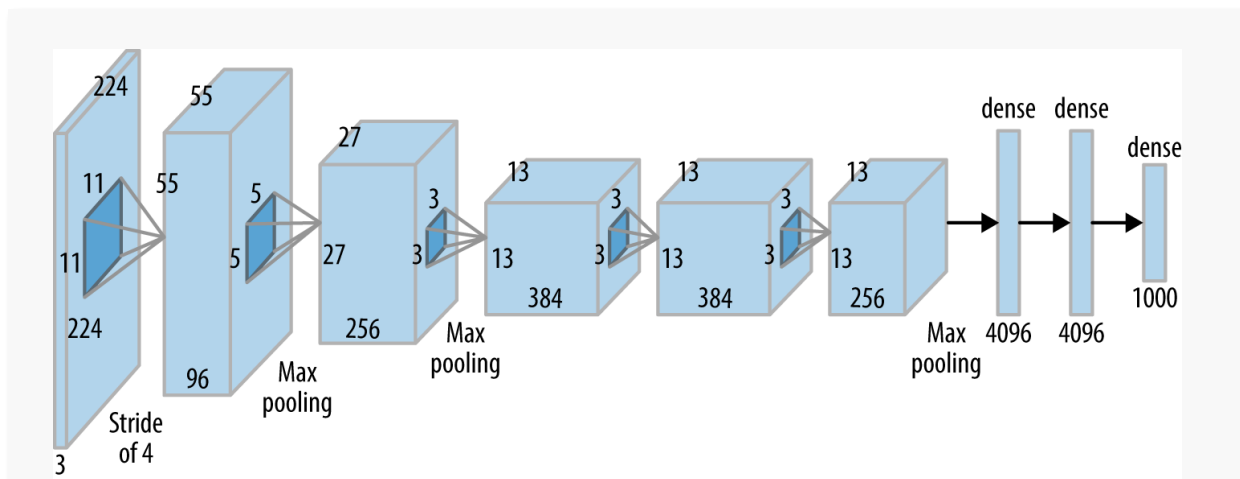


- **Coco** (<https://cocodataset.org/#home>)

- 330K de imágenes, anotadas (80 categorías, 5 descriptores por imagen)
- LVIS 2021 Challenge and Workshop



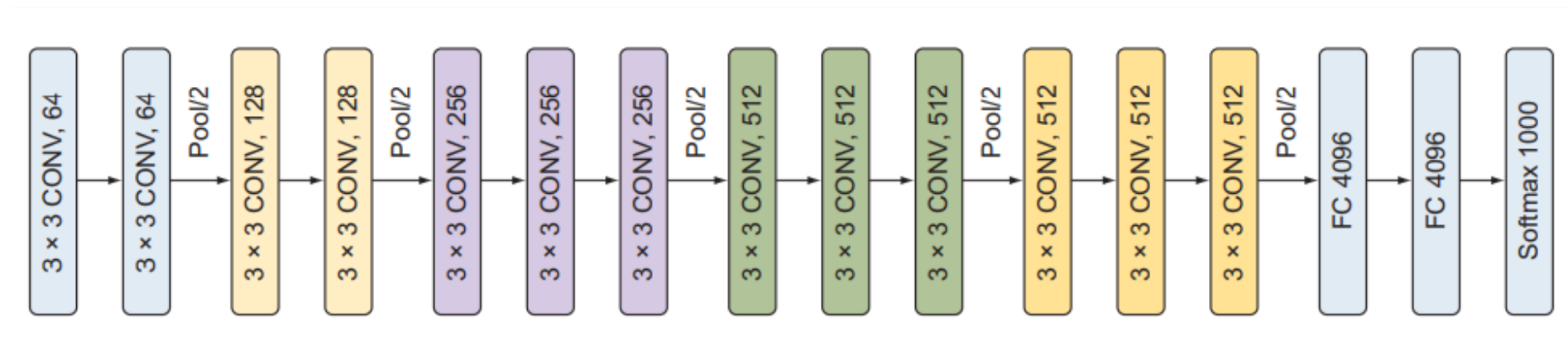
▪ AlexNet



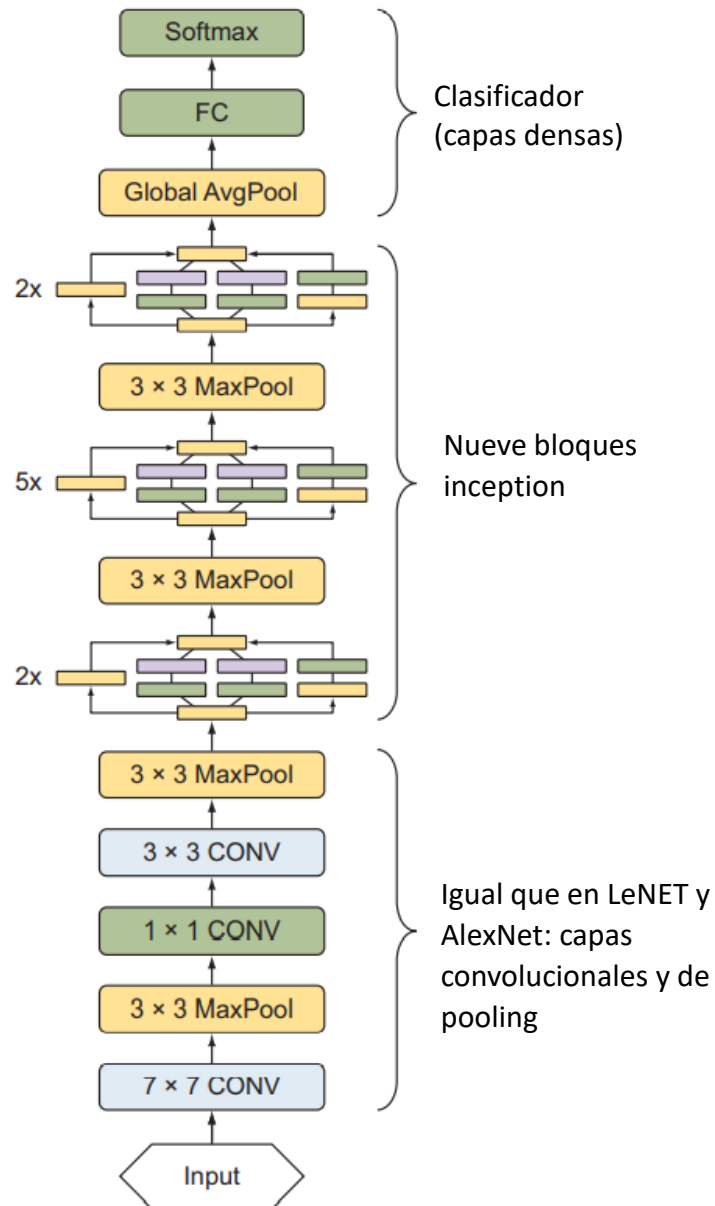
Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton (2012). ImageNet Classification with Deep Convolutional Neural Networks. Part of: Advances in Neural Information Processing Systems, 25.

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully connected	—	1,000	—	—	—	Softmax
F10	Fully connected	—	4,096	—	—	—	ReLU
F9	Fully connected	—	4,096	—	—	—	ReLU
S8	Max pooling	256	6 × 6	3 × 3	2	valid	—
C7	Convolution	256	13 × 13	3 × 3	1	same	ReLU
C6	Convolution	384	13 × 13	3 × 3	1	same	ReLU
C5	Convolution	384	13 × 13	3 × 3	1	same	ReLU
S4	Max pooling	256	13 × 13	3 × 3	2	valid	—
C3	Convolution	256	27 × 27	5 × 5	1	same	ReLU
S2	Max pooling	96	27 × 27	3 × 3	2	valid	—
C1	Convolution	96	55 × 55	11 × 11	4	valid	ReLU
In	Input	3 (RGB)	227 × 227	—	—	—	—

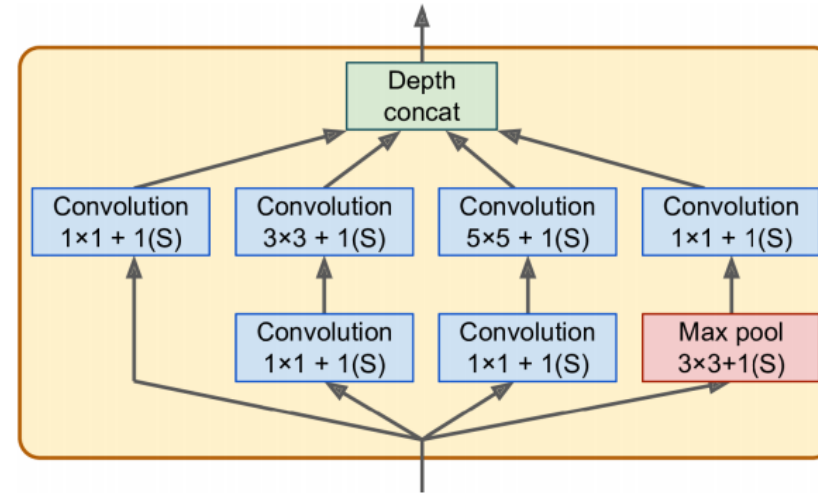
▪ VGGNet-16 (Visual Geometry Group (VGG), University of Oxford)



■ Inception (GoogLeNet)



Módulo Inception



- Las convoluciones de 1x1 aprenden patrones a través de la profundidad de la capa anterior.
- Las convoluciones de 3x3 y 5x5 aprenden patrones espaciales sobre la capa anterior.

Going deeper with convolutions.
(<https://arxiv.org/pdf/1409.4842v1.pdf>)

Otras:

- **ResNet**
- **Xception**
- **MbileNet**
- **DenseNet**
- **EfficientNet**

Investigar: ¿Qué son las conexiones residuales?

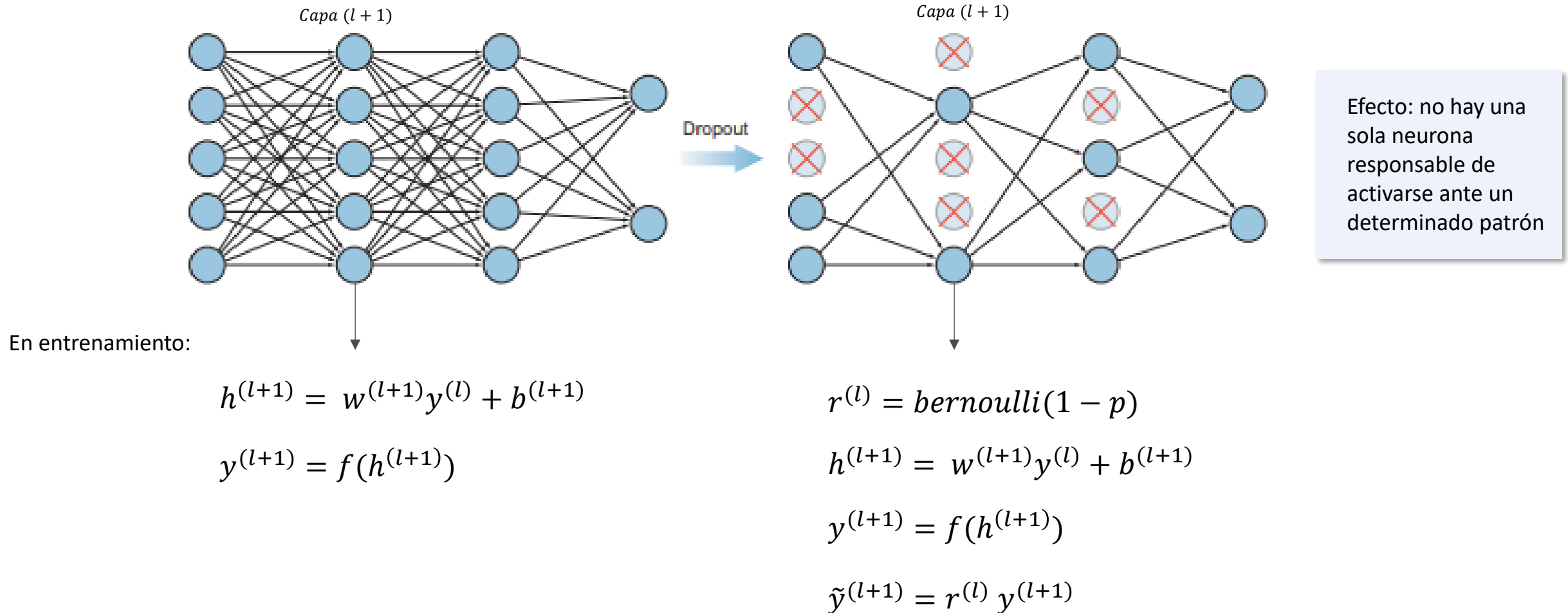
- ❑ **Dropout**
- ❑ **Aumentación de imágenes**
- ❑ **Aprendizaje por transferencia**

Además: ¿Cuáles hiperparámetros ajustar?

- Número de capas convolucionales.
- Número de mapas por capa.
- Tamaño de los filtros (Kernel).
- Tamaño del mini-batch,

Dropout

- Tipo de regularización. Ayuda a prevenir el sobreajuste.
- Durante el entrenamiento se "apagan" aleatoriamente un porcentaje de neuronas en una capa en cada iteración de entrenamiento. Durante el paso hacia adelante cada neurona tendrá una probabilidad p de ser "desactivada" (**dropout**).



Proceso:

Durante entrenamiento

- Aplicar dropout y desactivar una fracción de neuronas, con probabilidad p .
- Escalar las activaciones activas multiplicándolas por $\frac{1}{1-p}$

Durante la inferencia

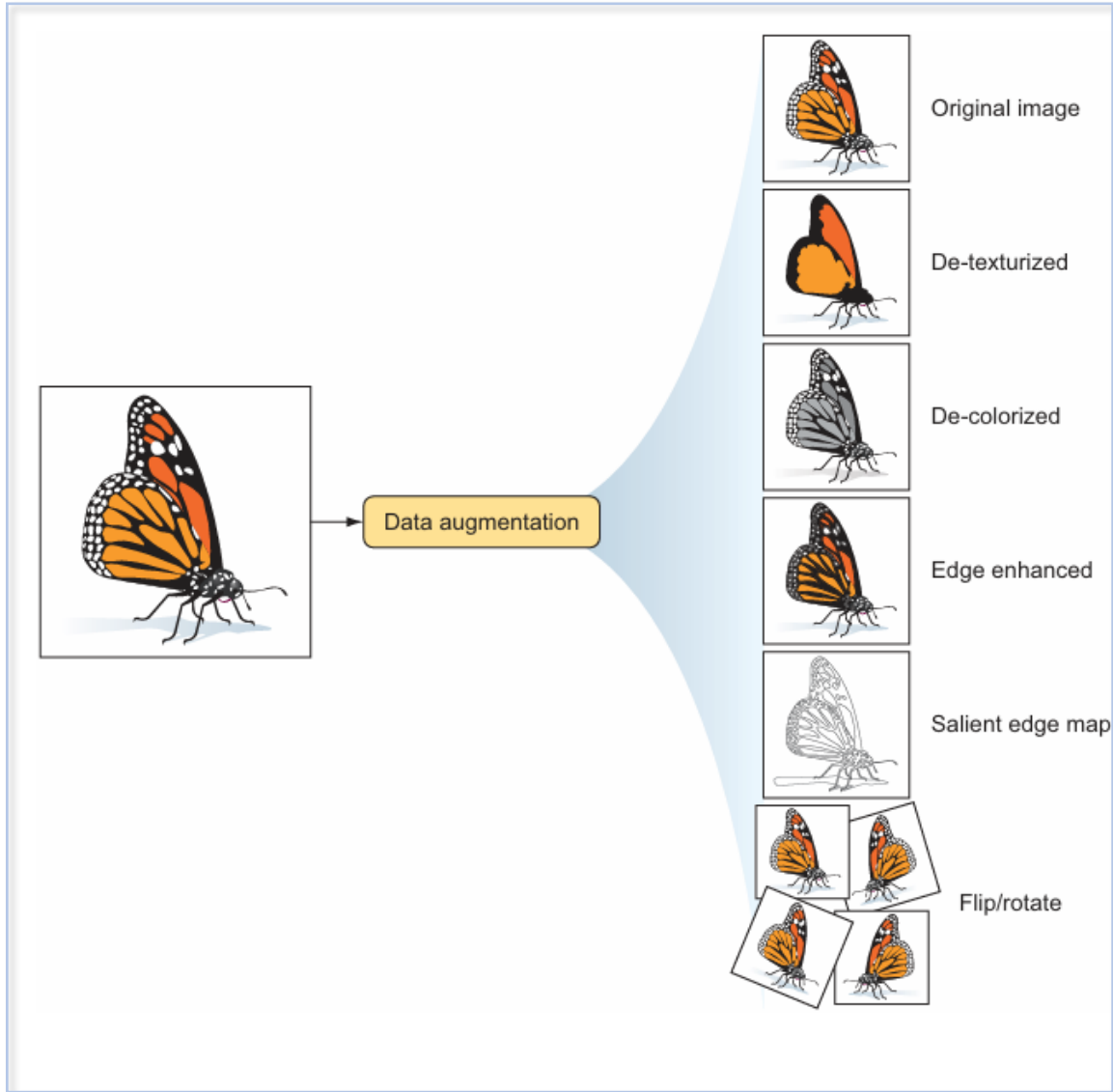
- No aplicar dropout.
- Utilizar todas las neuronas con las activaciones ya ajustadas.

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
                           kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
                           kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

En redes convolucionales

```
model = Sequential()
model.add(Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(64, 64, 3)))
model.add(Dropout(0.25))
model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
model.add(Dropout(0.25))
model.add(Conv2D(filters=128, kernel_size=(3, 3), activation='relu'))
model.add(Dropout(0.25))
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))
```

❏ Aumentación de imágenes.



```
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    horizontal_flip=True,  
    rotation_range=30,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    zoom_range=0.2  
)
```

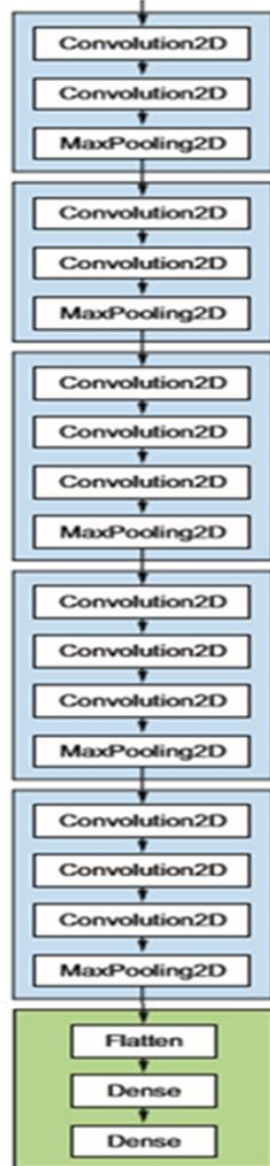
```
data_augmentation = models.Sequential([  
    layers.Rescaling(1./255),  
    layers.RandomFlip("horizontal"),  
    layers.RandomRotation(0.1),  
    layers.RandomTranslation(0.2, 0.2),  
    layers.RandomZoom(0.2),  
)
```



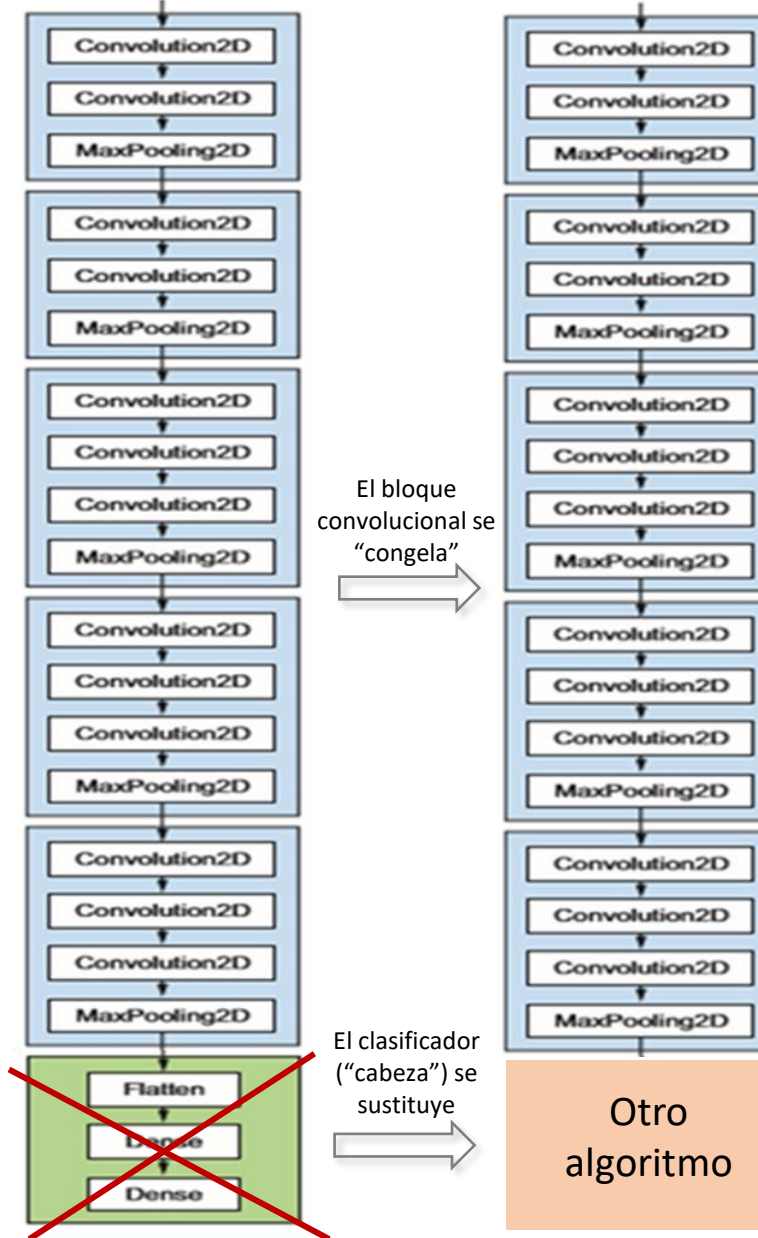
❑ Aprendizaje por transferencia (*transfer learning*)

- Transferencia de los conocimientos que la red ha adquirido resolviendo una tarea con una gran cantidad de datos, a una nueva tarea en la que los datos no son abundantes.
- Red pre-entrenada. Red que se ha entrenado previamente con un gran conjunto de datos, por ejemplo, en una tarea de clasificación de imágenes a gran escala.
- Se utilizan entonces redes pre-entrenadas (fundacionales) para hacer la transferencia de conocimiento
- Alternativas:

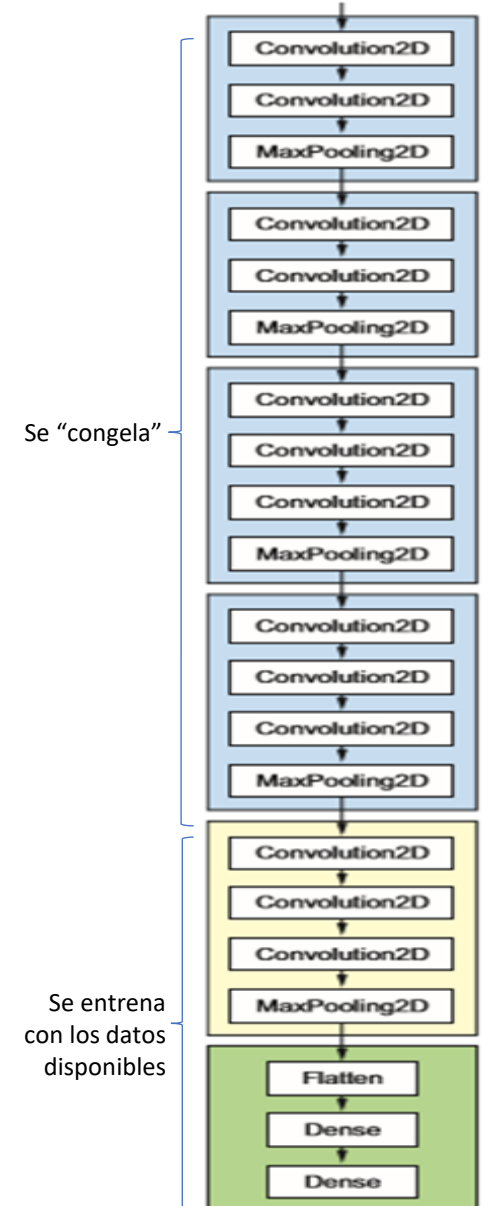
Usar la red completa para hacer predicciones



Utilizar el bloque convolucional como extractor de características



Mediante ajuste fino (*fine tuning*)



Próxima sesión.

- Autoencoders y GANs
- Modelos pre-entrenados.